周报5__常见链表题

已刷题目

力扣：简单×2; 中等×5； 困难×1;

洛谷：入门×1; 普及×1;

- 位运算
    1. 2166. 设计位集(**)
    2. 29. 两数相除(**)
- 链表
    1. 160. 相交链表(*)
    2. 25. K 个一组翻转链表(***)
    3. 138. 随机链表的复制(**)
    4. 234. 回文链表(*)
    5. 142. 环形链表 II(**)
    6. 148. 排序链表(**)
- 数组
    1. P5730 【深基5.例10】显示屏 (*)
    2. P1554 USACO06DEC梦中的统计 Dream Counting B (*)
    3. P2141[NOIP 2014 普及组] 珠心算测验(*)

学习知识点

1. `a / b` 结果向上取整技巧：

   `(a + b - 1) / b`;

2. 位图

   > 位图用一个 bit（位）来存储一个布尔值，而不是用一个字节（byte）或整数（int）。
   >
   > 通常使用整数数组来存储位：
   >
   > - 每个 `int` 有 32 位，可以存储 32 个布尔值；
   > - 对于 n 个元素，需要 `ceil(n/32)` 个整数，即 `(n + 31) / 32` 个整数；

3. 二进制实现四则运算

   > 加法：
   >
   > 1. 记录2数无进位相加，也就是两数按位异或；
   > 2. 记录进位信息，也就是两数按位与的结果左移1位；
   > 3. 再把无进位相加和进位信息相加，直到进位全为0；
   >
   > 减法:
   >
   > 1. a减b等于a和b的相反数相加

> 乘法：
>
> 1. b的i位为1，就让结果加上`a<<i`；
>
> 除法：
>
> 1. 保证a和b不为最小整数；
> 2. 让a和b绝对值x和y相除；
> 3. 从除符号位外的最高位开始依次判断，$x >= y \times 2^i$，说明结果的第i位为1

笔记

- 位运算
  - 2166. 设计位集(**)

    > 1. flip()不需要实际翻转每一位，使用一个"反转"标志`reverse`；
    > 2. 内部用`ones`记录1的数量，用`zeros`记录0的数量，反转就是交换0和1变量的值；
  - 29. 两数相除(**)

    > 1. 保证a和b不为最小整数；
    > 2. 让a和b绝对值x和y相除；
    > 3. 从除符号位外的最高位开始依次判断，$x >= y \times 2^i$，说明结果的第i位为1
- 链表
  - 25. K 个一组翻转链表(***)

    > 1. 遍历找到当前组的尾节点；
    > 2. 反转每一组的节点，反转后原来的尾节点成为头部，原来的头节点成为尾部

下周计划

代码

1）位运算

1. 2166. 设计位集(**)

```cpp
class Bitset {
    // 位集 Bitset 是一种能以紧凑形式存储位的数据结构
private:
    vector<int> myBitset;
    int size;
    int zeros;
    int ones;
    bool reverse;

public:
    Bitset(int n) {
        myBitset.resize((n + 31) / 32);
        size = n;
        zeros = n;
```

```
        ones = 0;
        reverse = false;
    }

    void fix(int idx) {
        int index = idx / 32;
        int bits = idx % 32;
        if (!reverse) {
            // 位图没反转过
            // 将idx位0修改为1
            if ((myBitset[index] & (1 << bits)) == 0) {
                // myBitset[index] |= 1 << bits;
                myBitset[index] ^= (1 << bits);
                zeros--;
                ones++;
            }
        } else {
            // 位图反转过
            // 将idx位1修改为0
            if ((myBitset[index] & (1 << bits)) != 0) {
                // myBitset[index] &= ~(1 << bits);
                myBitset[index] ^= (1 << bits);
                zeros--;
                ones++;
            }
        }
    }

    void unfix(int idx) {
        int index = idx / 32;
        int bits = idx % 32;
        if (!reverse) {
            // 位图没反转过
            // 将idx位1修改为0
            if ((myBitset[index] & (1 << bits)) != 0) {
                // myBitset[index] &= ~(1 << bits);
                myBitset[index] ^= (1 << bits);
                zeros++;
                ones--;
            }
        } else {
            // 位图反转过
            // 将idx位0修改为1
            if ((myBitset[index] & (1 << bits)) == 0) {
                // myBitset[index] |= 1 << bits;
                myBitset[index] ^= (1 << bits);
                zeros++;
                ones--;
            }
        }
    }

    void flip() {
```

```
        reverse = !reverse;
        int temp = ones;
        ones = zeros;
        zeros = temp;
    }

    bool all() {
        return ones == size;
    }

    bool one() {
        return ones > 0;
    }

    int count() {
        return ones;
    }

    string toString() {
        string s;
        int status, num, bits;
        for (int i = 0; i < size; i++) {
            num = myBitset[i / 32];
            bits = i % 32;
            status = (num >> bits) & 1;
            status ^= reverse ? 1 : 0; // 判断是否反转过
            s += status + '0'; // 转字符
        }
        return s;
    }
};

/**
 * Your Bitset object will be instantiated and called as such:
 * Bitset* obj = new Bitset(size);
 * obj->fix(idx);
 * obj->unfix(idx);
 * obj->flip();
 * bool param_4 = obj->all();
 * bool param_5 = obj->one();
 * int param_6 = obj->count();
 * string param_7 = obj->toString();
 */
```

2. 29. 两数相除(**)

```
class Solution {
    // 二进制实现四则运算
public:
    enum{MIN = INT_MIN};

    int divide(int dividend, int divisor) {
        int a = dividend, b = divisor;
```

```
        if (a == MIN && b == MIN) {
            return 1;
        } else if (a != MIN && b != MIN) {
            return div(a, b);
        } else if (b == MIN) {
            // a 不是整数最小，b是整数最小
            return 0;
        } else if (a == MIN && b == -1) {
            // 商严格大于 (2<<31) - 1
            return INT_MAX;
        }
        // a是整数最小，b不是整数最小，也不是-1
        // 让a加1个b或减一个b，再除以b，结果补偿1
        a = add(a, b < 0 ? neg(b) : b);
        int res = div(a, b);
        int offset = b < 0 ? 1 : neg(1);
        return add(res, offset);
    }

    int add(int a, int b) {
        int res = a;
        while (b != 0) {
            // a和b无进位相加
            res = a ^ b;
            b = (a & b) << 1;
            a = res;
        }
        return res;
    }

    // 取补码
    int neg(int a) {
        return add(~a, 1);
    }

    int minus(int a, int b) {
        return add(a, neg(b));
    }

    int multiply(int a, int b) {
        // 龟速乘法
        int res = 0;
        while (b != 0) {
            if ((b & 1) != 0) {
                a = add(res, a);
            }
            a <<= 1;
            b = (unsigned int) b >> 1;
        }
        return res;
    }

    int div(int a, int b) {
```

```
        // 必须保证a和b都不是整数最小值，返回a除以b的结果
        int x = a < 0 ? neg(a) : a;
        int y = b < 0 ? neg(b) : b;
        int res = 0;
        // 分解x 为 (y << a) + (y << b) + …… + (y << 1) + c
        for (int i = 30; i >= 0; i--) {
            // x >= y << i; x 大于等于y乘2的i次方
            if ((x >> i) >= y) {
                res |= (1 << i);
                x = minus(x, y << i);
            }
        }
        // a和b不同号，则结果加负号
        return a < 0 ^ b < 0 ? neg(res) : res;
    }
};
```

2）链表

1. 160. 相交链表(*)

> 给你两个单链表的头节点 `headA` 和 `headB` ，请你找出并返回两个单链表相交的起始节点。如果两个链表不存在相交节点，返回 `null` 。

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode *getIntersectionNode(ListNode *headA, ListNode *headB) {
        if (headA == nullptr || headB == nullptr) {
            return nullptr;
        }
        ListNode* a = headA;
        ListNode* b = headB;
        int diff = 0;
        while (a != nullptr) {
            a = a->next;
            diff++;
        }
        while (b != nullptr) {
            b = b->next;
            diff--;
        }
        // 若最后节点不等，说明没有相交
```

```
        if (a != b) {
            return nullptr;
        }

        // a指向较长节点的头
        if (diff >= 0) {
            a = headA;
            b = headB;
        } else {
            a = headB;
            b = headA;
        }

        // 取长度差绝对值，让较长节点先走
        diff = abs(diff);
        for (int i = 0; i < diff; i++) {
            a = a->next;
        }
        while (a != b) {
            a = a->next;
            b = b->next;
        }
        return a;
    }
};
```

2. 25. K 个一组翻转链表(***)

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* reverseKGroup(ListNode* head, int k) {
        ListNode* start = head;
        ListNode* end = teamEnd(head, k);
        if (end == nullptr) {
            return head; // 只有一个节点，不反转
        }
        // 第一组节点尾部反转后为头
        head = end;
        reverse(start, end);
        // 反转后start变为上一组结尾节点
        ListNode* preTeamTail = start;
        while (preTeamTail->next != nullptr) {
            start = preTeamTail->next;
```

```
            end = teamEnd(start, k);
            if (end == nullptr) {
                return head;
            }
            reverse(start, end);
            // 反转后把上一组尾与当前组连上
            preTeamTail->next = end;
            // 处理完当前组节点
            preTeamTail = start;
        }
        return head;
    }

    // 找当前组尾节点
    ListNode* teamEnd(ListNode* curSt, int k) {
        for(int i = 1; i < k && curSt != nullptr; i++) {
            curSt = curSt->next;
        }
        return curSt;
    }

    // 反转一组节点
    void reverse(ListNode* curhead, ListNode* curtail) {
        // 下一组头节点
        curtail = curtail->next;
        ListNode* pre = nullptr;
        ListNode* cur = curhead;
        ListNode* next = nullptr;
        while (cur != curtail) {
            // 当前组的节点没处理完
            // 则处理节点
            next = cur->next; // 记录下一节点
            cur->next = pre; // 反转链表
            pre = cur;
            cur = next;
        }
        curhead->next = curtail;
    }
};
```

3. 138. 随机链表的复制(**)

> 给你一个长度为 `n` 的链表，每个节点包含一个额外增加的随机指针 `random` ，该指针可以指向链表中的任何节点或空节点。
>
> 构造这个链表的 **深拷贝**。 深拷贝应该正好由 `n` 个 **全新** 节点组成，其中每个新节点的值都设为其对应的原节点的值。新节点的 `next` 指针和 `random` 指针也都应指向复制链表中的新节点，并使原链表和复制链表中的这些指针能够表示相同的链表状态。**复制链表中的指针都不应指向原链表中的节点** 。

```
/*
// Definition for a Node.
class Node {
public:
```

```
    int val;
    Node* next;
    Node* random;

    Node(int _val) {
        val = _val;
        next = NULL;
        random = NULL;
    }
};
*/

class Solution {
public:
    Node* copyRandomList(Node* head) {
        if (head == nullptr) return nullptr;
        // 节点的指针
        Node* cur = head;
        Node* next = nullptr;

        // 复制节点，并建立其next为复制节点的对应关系
        while (cur != nullptr) {
            // 复制节点接在cur和next间
            next = cur->next;
            cur->next = new Node(cur->val);
            cur = cur->next; // 来到复制的节点处
            cur->next = next;
            cur = next; // 前进到next节点
        }

        // 复制random指针关系
        cur = head;
        Node* copy = nullptr;
        while (cur != nullptr) {
            // 记录节点
            copy = cur->next;
            next = copy->next;
            // 调整random
            copy->random = cur->random != nullptr ? cur->random->next : nullptr;
            cur = next;
        }

        // 分离原节点和新节点
        cur = head;
        Node* res = cur->next;
        while(cur != nullptr) {
            // 记录节点
            copy = cur->next;
            next = copy->next;
            // 调整copy的next指针
            copy->next = next != nullptr ? next->next : nullptr;
            cur->next = next;
            cur = next; // 移到next节点
```

```
        }
        return res;
    }
};
```

4. 234. 回文链表(*)

> 判断链表是不是回文结构

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    bool isPalindrome(ListNode* head) {
        if (head == nullptr || head->next == nullptr) {
            return true;
        }

        // 快慢指针找中点
        ListNode* slow = head;
        ListNode* fast = head;
        while (fast->next != nullptr && fast->next->next != nullptr) {
            fast = fast->next->next;
            slow = slow->next;
        }
        // 反转中点slow右边链表
        ListNode* pre = slow;
        ListNode* cur = pre->next;
        ListNode* next = nullptr;
        pre->next = nullptr;
        while (cur != nullptr) {
            next = cur->next;
            cur->next = pre;
            pre = cur;
            cur = next;
        }

        bool ans = true;
        ListNode* left = head;
        ListNode* right = pre;
        while (left != nullptr && right != nullptr) {
            if (left->val != right->val) {
                ans = false;
                break;
            }
```

```
            left = left->next, right = right->next;
        }

        // 恢复链表
        cur = pre->next;
        pre->next = nullptr; // 调整指针，让链表末尾指向空
        while (cur != nullptr) {
            next = cur->next;
            cur->next = pre;
            pre = cur;
            cur = next;
        }

        return ans;
    }
};
```

5. 142. 环形链表 II(**)

> 找到环形链表的入环节点
>
> 快慢指针

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode *detectCycle(ListNode *head) {
        if (head == nullptr || head->next == nullptr || head->next == nullptr) {
            return nullptr;
        }

        // 快慢指针相遇就是有环链表
        // 不相遇则是无环链表
        ListNode* slow = head->next;
        ListNode* fast = head->next->next;
        while (fast != slow) {
            if (fast == nullptr || fast->next == nullptr || fast->next->next ==
nullptr) {
                return nullptr;
            }
            fast = fast->next->next;
            slow = slow->next;
        }

        // fast指针从头开始，再次相遇的节点即使入环节点
        fast = head;
        while (fast != slow) {
```

```
            fast = fast->next;
            slow = slow->next;
        }
        return slow;
    }
};
```

6. 148. 排序链表(**)

> 链表的归并排序
>
> 时间复杂度$O(N*logN)$;
>
> 空间复杂度$O(1)$;

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* sortList(ListNode* head) {
        // 长度小于2，天然有序
        if(head == nullptr || head->next == nullptr) {
            return head;
        }

        int n = 0;
        ListNode* cur = head;
        while (cur != nullptr) {
            cur = cur->next;
            n++;
        }

        // 迭代链表归并排序
        ListNode* next, *lastTeamEnd, *lh, *lt, *rh, *rt;
        // 从每组左右长度各为一开始二路归并
        // step为每一组左部的长度
        for (int step = 1; step < n; step <<= 1) {
            // 第一组单独处理，因为可能改变链表头
            lh = head;
            lt = findEnd(lh, step);
            rh = lt->next;
            rt = findEnd(rh, step);
            next = rt->next;
            // 断连
            lt->next = nullptr;
            rt->next = nullptr;
```

```
            merge(lh, lt, rh, rt);
            head = start;
            lastTeamEnd = end;

            // 迭代归并后序的组
            while (next != nullptr) {
                lh = next;
                lt = findEnd(lh, step);
                rh = lt->next;
                if (rh == nullptr) {
                    // 上一组尾接这一组头
                    lastTeamEnd->next = lh;
                    break;
                }
                rt = findEnd(rh, step);
                next = rt->next;
                lt->next = nullptr;
                rt->next = nullptr;
                merge(lh, lt, rh, rt);
                // 上一组尾接这一组头
                lastTeamEnd->next = start;
                lastTeamEnd = end;
            }
        }
        return head;
    }

    // 包括头在内，往下数k个节点
    ListNode* findEnd(ListNode*head, int k) {
        for (int i = 1; i < k && head->next != nullptr; i++) {
            head = head->next;
        }
        return head;
    }

    // 全局变量记录整组的头
    ListNode* start, *end;
    // 归并两个有序链表
    void merge(ListNode* lh, ListNode* lt, ListNode* rh, ListNode* rt) {
        ListNode* pre;
        if (lh->val <= rh->val) {
            start = lh;
            pre = lh;
            lh = lh->next;
        } else {
            start = rh;
            pre = rh;
            rh = rh->next;
        }

        // 未越界时
        while (lh != nullptr && rh != nullptr) {
            if (lh->val <= rh->val) {
```

```
                pre->next = lh;
                pre = lh;
                lh = lh->next;
            } else {
                pre->next = rh;
                pre = rh;
                rh = rh->next;
            }
        }

        // 右链表走完
        if (lh != nullptr) {
            pre->next = lh;
            end = lt;
        } else {
        // 左链表走完
            pre->next = rh;
            end = rt;
        }
    }
};
```

3）数组

1. P5730 【深基5.例10】显示屏 (*)

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    vector<string> patterns[10] = {
        // 0
        {
        "XXX",
        "X.X",
        "X.X",
        "X.X",
        "XXX"},
        // 1
        {
        "..X",
        "..X",
        "..X",
        "..X",
        "..X"},
```

```
        // 2
        {
        "xxx",
        "..x",
        "xxx",
        "x..",
        "xxx"},
        // 3
        {
        "xxx",
        "..x",
        "xxx",
        "..x",
        "xxx"},
        // 4
        {
        "x.x",
        "x.x",
        "xxx",
        "..x",
        "..x"},
        // 5
        {
        "xxx",
        "x..",
        "xxx",
        "..x",
        "xxx"},
        // 6
        {
        "xxx",
        "x..",
        "xxx",
        "x.x",
        "xxx"},
        // 7
        {
        "xxx",
        "..x",
        "..x",
        "..x",
        "..x"},
        // 8
        {
        "xxx",
        "x.x",
        "xxx",
        "x.x",
        "xxx"},
        // 9
        {
        "xxx",
        "x.x",
```

```
        "XXX",
        "..X",
        "XXX"}
    };

    int n;
    string digits;
    cin >> n >> digits;
    // 逐行输出
    for (int row = 0; row < 5; row++) {
        string line;
        for (char c: digits) {
            // 字符数字转为整数索引
            int d = c - '0';
            line += patterns[d][row];
            line += '.'; // 打印数字间隔
        }

        // 去除最后一个多余的间隔
        if (!line.empty()) {
            line.pop_back();
        }
        cout << line << '\n';
    }
    return 0;
}
```

2. P1554 USACO06DEC梦中的统计 Dream Counting B (*)

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int M, N;
    cin >> M >> N;
    vector<int> cnts(10, 0);

    for (int i = M; i <= N; i++) {
        int num = i;
        // 单独统计为0的数
        if (num == 0) {
            cnts[num]++;
            continue;
        }
        // 逐位统计出现的数字
        while (num > 0) {
            int bit = num % 10;
            cnts[bit]++;
```

```
            num /= 10;
        }
    }

    for (int i = 0; i < 10; i++) {
        cout << cnts[i];
        if (i != 9) {
            cout << ' ';
        } else {
            cout << '\n';
        }
    }
}
```

3. P2141[NOIP 2014 普及组] 珠心算测验(*)

```
#include "iostream"
#include "vector"

int main() {
    using namespace std;
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // 判断符合条件的数
    int count = 0;
    for (int num: arr) {
        bool isSum = false;
        for (int i = 0; i < n && !isSum; i++) {
            for (int j = i + 1; j < n && !isSum; j++) {
                if (num == arr[i] + arr[j]) {
                    count++;
                    isSum = true;
                    break;
                }
            }
        }
    }
    cout << count << '\n';
    return 0;
}
```